



UNITS 1- 3

MID-TERM EXAM REVIEW





UNIT I

INTRODUCTION TO PROGRAMMING



CLASS STANDARDS

- Indentation
- Line length
- Comments
- Constant declarations
- Variable declarations
- Calculations and Variables
- Naming Conventions
- Semicolons
- Whitespace

VARIABLES

- Storage space, like a container, that holds data.
- The “container” holds a reference to a place in RAM (Random Access Memory) where that data is being stored.

//Variable declaration

```
let myName;
```

//Variable assignment

```
myName = “Jane”;
```

DATA TYPES

- **Numbers** (5, -10, 4.2, 7e10)
- **Strings** (always enclosed by 'single' or "double" quotes)
- **Booleans** (true or false)
- **Null** (an empty variable)
- **Undefined** (variable has not been declared, or has not been given a value)
- **Objects** (we've seen both the window and document objects)

ESCAPE CHARACTERS

Remember to use the **BACK**slash.

`\b` Backspace

`\t` tab

`\n` new line

`\v` vertical tab

`\f` form feed

`\r` carriage return

`\\` literal backslash

`\"` double quote

`\'` single quote

METHODS

//Window object methods

alert();

prompt(); //always returns a String!

//Document object methods

document.write();

THE **LENGTH** PROPERTY

- Useful property of a String
- **No parenthesis!**

//You can **only** use length property on a String

```
let myName = "Jane";
```

```
document.write(myName.length);
```




UNIT 2

ALL ABOUT OPERATORS



ASSIGNMENT OPERATORS

```
let totalPrice;  
totalPrice = 10;    //totalPrice is 10  
totalPrice += 5;    //totalPrice is 15  
totalPrice -= 3;    //totalPrice is 12  
totalPrice *= 2;    //totalPrice is 24  
totalPrice /= 8;    //totalPrice is 3
```

`-=` `*=` `/=`

Will attempt to
convert Strings to
Numbers.

`+=`

Will concatenate
if one value is a
String

ADDITIVE OPERATORS

```
let totalPrice = 10;
```

```
let answer;
```

```
//Numbers
```

```
answer = totalPrice + 10;    //answer is 20
```

```
answer = totalPrice - 5;    //answer is 15
```

```
//Strings
```

```
let totalPrice = 10;
```

```
answer = totalPrice + "2";  //answer is 102
```

```
answer = totalPrice - "2";  //answer is 8
```

+

Will concatenate
if one value is a
String

MULTIPLICATION & DIVISION OPERATORS

```
let totalPrice = 10;
```

```
let answer;
```

```
//Numbers
```

```
answer = totalPrice * 10;    //answer is 100
```

```
answer = totalPrice / 5;    //answer is 2
```

```
//Strings
```

```
let totalPrice = 10;
```

```
answer = totalPrice * "10";  //answer is 100
```

```
answer = totalPrice / "5";   //answer is 2
```

* /

Will always
attempt to
convert a String to
a Number

EQUALITY OPERATORS

Attempts to convert Strings to number before comparison

`==` //is equal to `!=` //is not equal to

No conversions, **STRICT** comparison

`===` //is equal in **value** and **data type**

`!==` //is not equal in **value** and **data type**

`==` `!=`
`===` `!==`

Will always
return a Boolean
(true or false)

EQUALITY OPERATORS

```
let totalPrice = 10;
let answer;

answer = totalPrice == 10;    //answer is true
answer = totalPrice != 10;    //answer is false
answer = totalPrice === 15;   //answer is false
answer = totalPrice !== 15;   //answer is true
```

=

Does **NOT**
mean equality!

== !=

=== !==

Will always
return a Boolean
(true or false)

RELATIONAL OPERATORS WITH NUMBERS

```
let totalPrice = 10;
let answer;

answer = totalPrice > 10;    //answer is false
answer = totalPrice >= 10;   //answer is true
answer = totalPrice < 15;    //answer is true
answer = totalPrice <= 15;   //answer is true
```

> < >= <=

Will always
return a Boolean
(true or false)

**NEVER USE RELATIONAL OPERATORS TO COMPARE NUMBERS
AGAINST STRINGS**

RELATIONAL OPERATORS WITH STRINGS

```
let myName = "Sally";  
let answer;  
answer = myName > "Nick";    //answer is true  
answer = myName >= "8";      //answer is true  
answer = myName < "Sam";     //answer is true  
answer = myName <= "Sally";  //answer is true
```

> < >= <=

Will always
return a Boolean
(true or false)

**NEVER USE RELATIONAL OPERATORS TO COMPARE NUMBERS
AGAINST STRINGS**

ASCII CHARACTER ORDER

SMALLEST

LARGEST



Digits (48-57)

UPPERCASE LETTERS (65-90)

lowercase letters (97-122)

UNARY OPERATORS

`++` increments a Number by 1

`--` decrements a Number by 1

```
let my count = 5;
```

```
myCount++;           //myCount is 6
```

```
myCount--;           //myCount is 5
```

```
typeof(myCount);     //"number"
```

`++` `--`

Will always
attempt to
convert a String to
a Number

UNARY OPERATORS CONT.

`typeof()` returns a string
representing the data type

```
typeof(5);           //number  
typeof("George");    //string  
typeof(20 < 56);     //boolean
```

OPERATOR PRECEDENCE

HIGHEST

•

()

++ --

typeof

* / %

+ -

< <= > >=

== != === !==

LOWEST

= += -= *= /=

Always use parentheses when doing math with more than two parts.

If operators have the same precedence will be performed from left to right.



UNIT 3

ALL ABOUT IF



SIMPLE IF STATEMENT

```
if (count === 5) {  
    document.write("count is equal to 5");  
}
```

SIMPLE IF STATEMENT

```
if (count === 5) {
```

The condition in the parentheses must return true or false

```
document.write("count is equal to 5");
```

```
}
```

If the condition is true, the code between the curly braces will run

IF / ELSE STATEMENT

```
if (count === 5) {  
    document.write("count is equal to 5");  
} else {  
    document.write("count is not 5");  
}
```


IF / ELSE STATEMENT

```
if (count === 5) {
```

```
    document.write("count is equal to 5");
```

```
} else {
```

```
    document.write("count is not 5");
```

```
}
```

If the condition is true,
the code after the if
curly braces will run

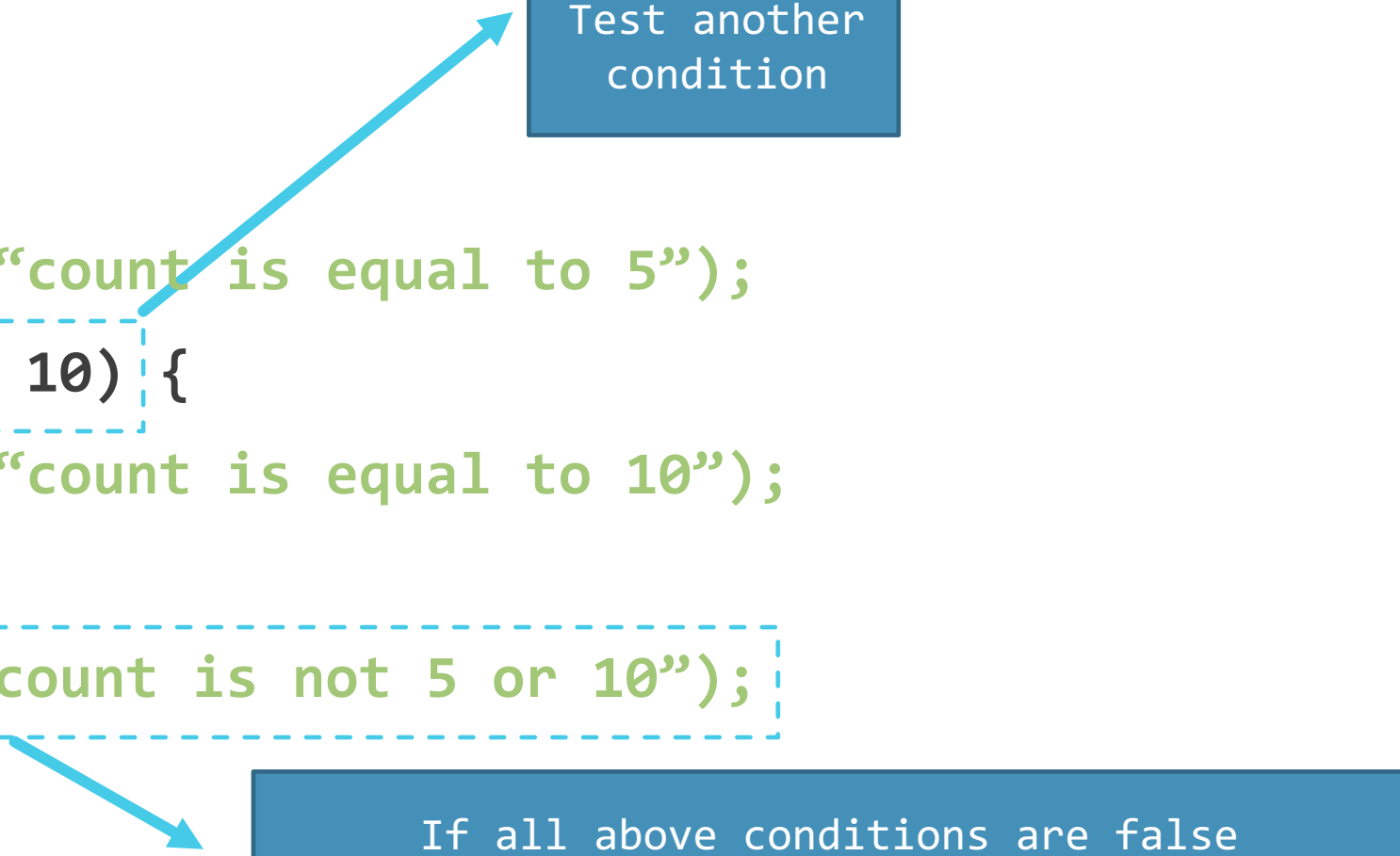
If the condition is false,
the code after the else
curly braces will run

IF / ELSE IF STATEMENT

```
if (count === 5) {  
    document.write("count is equal to 5");  
} else if (count === 10) {  
    document.write("count is equal to 10");  
} else {  
    document.write("count is not 5 or 10");  
}
```

IF / ELSE IF STATEMENT

```
if (count === 5) {  
    document.write("count is equal to 5");  
} else if (count === 10) {  
    document.write("count is equal to 10");  
} else {  
    document.write("count is not 5 or 10");  
}
```

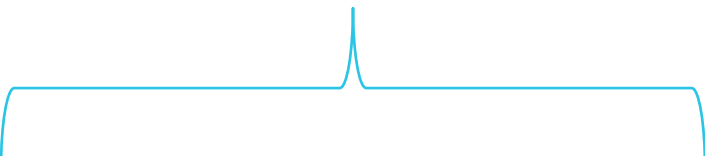


Test another
condition

If all above conditions are false

LOGICAL OR

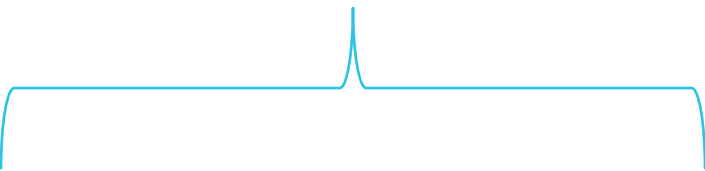
One of the conditions must be true for the code between the curly braces to run



```
if (count === 5 || count === 6) {  
    document.write("That is the right amount");  
}
```

LOGICAL AND

Both of the conditions must be true for the code between the curly braces to run



```
if (count === 5 && count === 6) {  
    document.write("That is the right amount");  
}
```